ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО
ОБРАЗОВАНИЯ

РОССИЙСКИЙ УНИВЕРСИТЕТ ДРУЖБЫ НАРОДОВ
ИМЕНИ ПАТРИСА ЛУМУМБЫ

Факультет искусственного интеллекта

Отчет

о выполнении лабораторной работы № 1

“Персептрон Розенблатта”

по дисциплине "Нейронные сети"

Вариант задания № 13

Выполнил студент группы ЗФИбд-01-24

Князев А. М._____

Проверил и принял

доцент кафедры прикладного ИИ Леонов С.С._____

с оценкой _____

Москва, 2026

Цель работы: исследование свойств персептрона Розенблатта и его применение для решения задачи распознавания образов (классификации точек на плоскости).

Основные этапы работы:

1. Сформировать первую обучающую выборку из шести точек плоскости в диапазоне [−5; 5], разделённых на два линейно разделимых класса. Построить однонейронный персептрон Розенблатта с пороговой функцией активации hardlim и обучить его по правилу Розенблатта: сначала вручную за две эпохи с пошаговым пересчётом весов и смещения, затем автоматически до достижения нулевой ошибки. Отобразить полученную дискриминантную прямую и проверить качество классификации на контрольных точках области.
2. Добавить в обучающую выборку точку, нарушающую линейную разделимость классов, и повторить обучение по правилу Розенблатта. Установить, что в этом случае итеративная коррекция весов не сходится к нулевой ошибке, оценить итоговую среднюю абсолютную ошибку (MAE) и сделать вывод о принципиальном ограничении однослойного персептрона.
3. Сформировать вторую обучающую выборку из восьми точек, распределённых по четырём классам, и закодировать классы двухбитным выходом двух нейронов. Построить и обучить такой персептрон по правилу Розенблатта с перебором случайных начальных приближений до нулевой ошибки, отобразить две разделяющие прямые и проверить отнесение контрольных точек к четырём областям плоскости.

Программное обеспечение: Python 3.x, Jupyter Notebook, библиотеки NumPy и Matplotlib.

Сценарий выполнения работы

Персептрон Розенблатта и правило обучения

Персептрон Розенблатта — однослойная нейронная сеть. Каждый нейрон вычисляет взвешенную сумму входов со смещением $n = W \cdot x + b$ и применяет пороговую функцию активации hardlim: выход равен 1 при $n \geq 0$ и 0 при $n < 0$. Для бинарной классификации используется один нейрон с

двумя входами (x1, x2) и смещением, для классификации на четыре класса — два нейрона, выходы которых задают двухбитный код класса.

Обучение выполняется по правилу Розенблатта. Для каждого образца вычисляется выход сети y, ошибка $e = t - y$ (t — целевой выход) и при ненулевой ошибке веса и смещение корректируются: $W \leftarrow W + \eta \cdot e \cdot x^T$, $b \leftarrow b + \eta \cdot e$, где η — скорость обучения (взята $\eta = 0.5$). Образцы предъявляются циклически по эпохам; обучение завершается при достижении нулевой ошибки на всей выборке (MAE = 0) либо по достижении предельного числа эпох. Начальные веса и смещение задаются случайными значениями из равномерного распределения $[-1, 1]$.

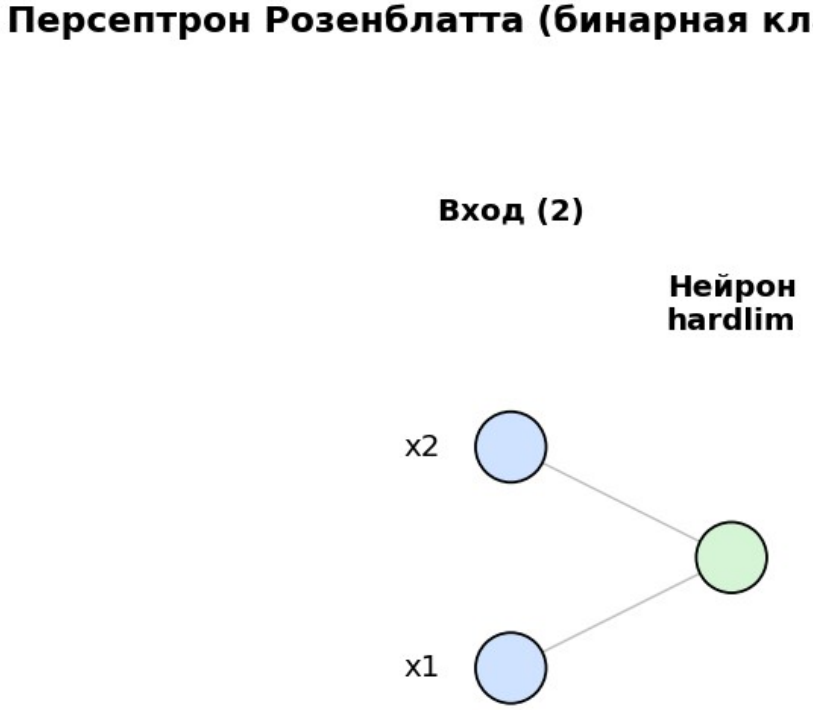


Рис. 1 — Схема персептрона Розенблатта (бинарная классификация)

Этап 1. Бинарная классификация (2 класса, 6 точек)

Задана обучающая выборка из 6 точек плоскости в диапазоне [−5; 5], разделённых на два класса. Координаты точек и метки приведены в таблице 1.

Таблица 1 — Обучающая выборка (бинарная классификация)

№	x1	x2	Класс
1	0.7	−4.5	0
2	−2.7	−1.5	1
3	3.2	−4.9	0
4	−4.6	−3.4	1
5	1.4	2.3	1
6	1.4	−0.5	0

Ручное обучение (2 эпохи). Веса и смещение инициализированы случайно (seed = 12): W = [−0.692, 0.480], b = −0.473. На каждой эпохе образцы предъявляются по очереди и при ошибке корректируются веса по правилу Розенблатта. После первой эпохи MAE = 0.333, после второй MAE = 0.000 — сеть полностью разделила классы.

Таблица 2 — Ход ручного обучения

Эпоха	W	b	MAE
1	[0.008, 1.630]	0.027	0.333
2	[−1.342, 0.880]	0.527	0.000

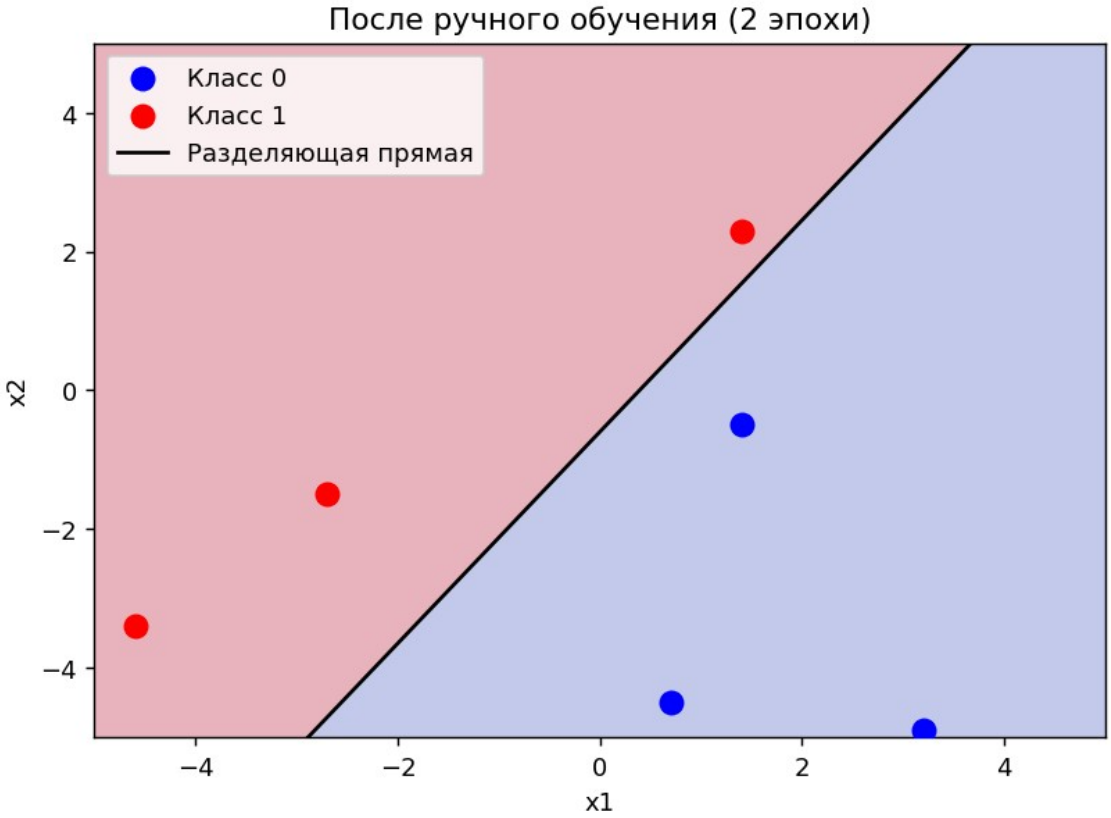


Рис. 2 — Результат после ручного обучения (2 эпохи)

Автоматическое обучение (до 50 эпох) с другой случайной инициализацией (seed = 15) также сходится к нулевой ошибке. Итоговые параметры: W = [−1.652, 1.258], b = 0.109, MAE = 0.000. Для проверки качества классифицированы 3 случайные точки области — все отнесены к классам в соответствии с построенной дискриминантной прямой.

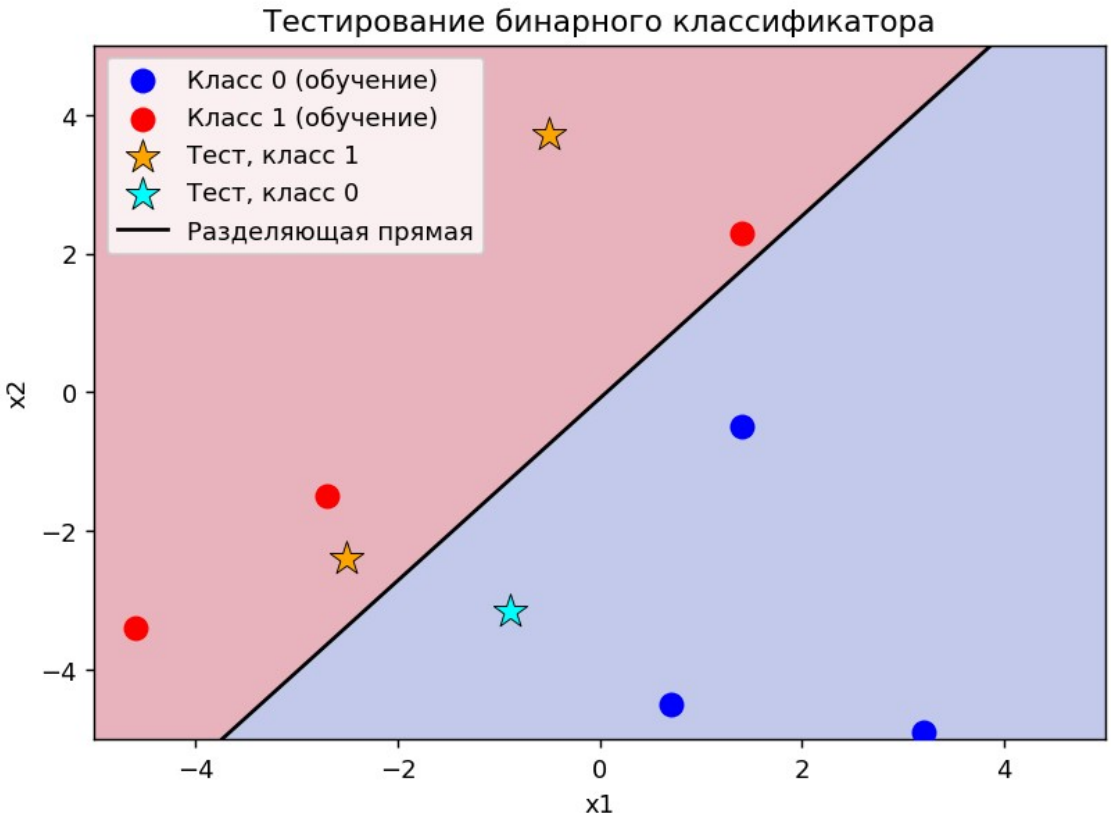


Рис. 3 — Тестирование бинарного классификатора (3 контрольные точки)

Этап 2. Линейно неразделимый случай

В обучающую выборку добавлена точка (−3.0, 1.0) класса 0, расположенная внутри области класса 1. После этого классы становятся линейно неразделимыми: ни одна прямая не разделяет их без ошибок. Обучение по правилу Розенблатта не сходится к нулевой ошибке — итоговая MAE = 0.4286 (в итоговом состоянии неверно классифицируются три точки из семи), веса колеблются и не стабилизируются. Это демонстрирует фундаментальное ограничение однослойного персептрона: он способен разделять только линейно разделимые классы.

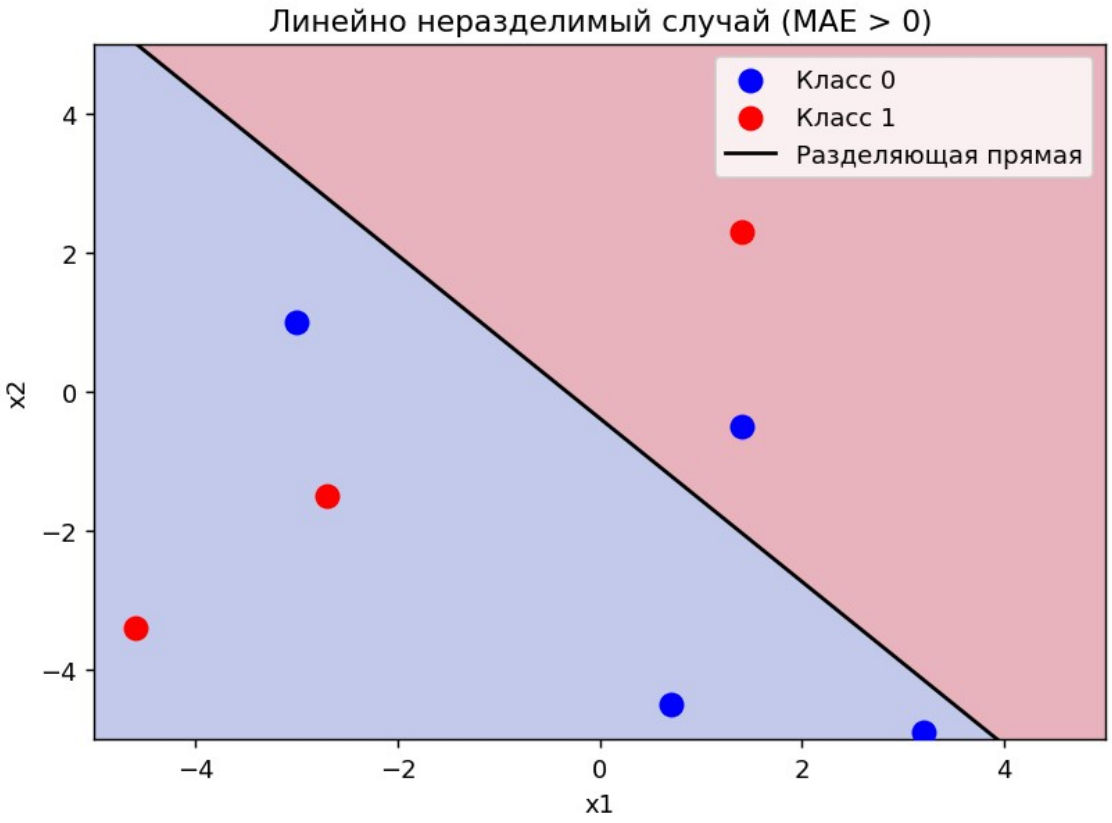


Рис. 4 — Линейно неразделимый случай (MAE > 0)

Этап 3. Классификация на 4 класса (2 нейрона)

Задана вторая обучающая выборка из 8 точек. Четыре класса кодируются двухбитным выходом двух нейронов: [0,0], [0,1], [1,0] и [1,1]. Координаты точек и целевые двухбитные метки приведены в таблице 3.

Таблица 3 — Обучающая выборка (4 класса)

№	x1	x2	Метка [t1, t2]
1	0.6	−2.4	[1, 0]
2	2.4	0.0	[1, 1]
3	1.4	−2.0	[1, 0]
4	−3.7	−0.3	[0, 0]
5	−1.4	2.8	[0, 1]
6	2.8	1.6	[1, 1]
7	−3.7	−4.8	[1, 0]
8	0.5	−2.0	[1, 0]

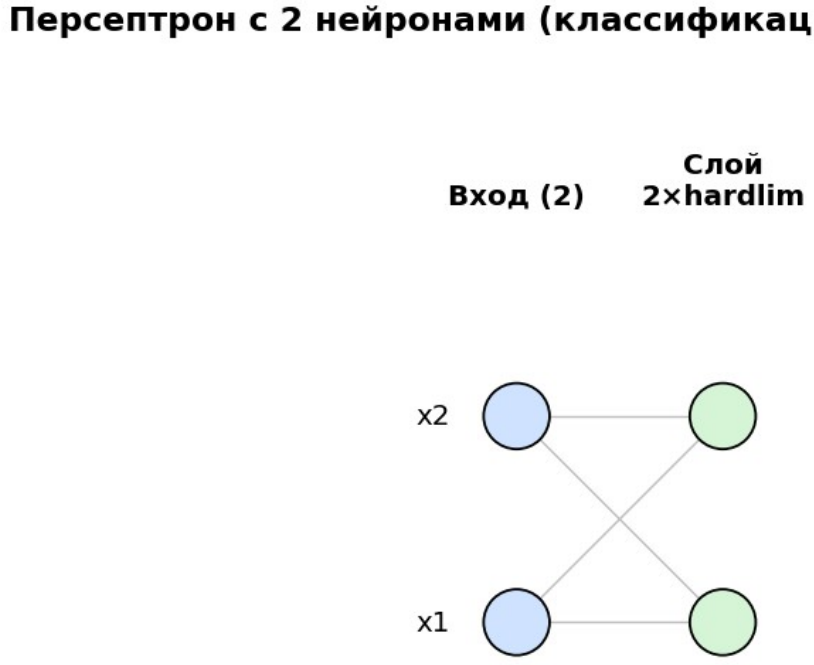


Рис. 5 — Схема персептрона с двумя нейронами (4 класса)

Сеть из двух нейронов обучается по тому же правилу Розенблатта.

Обучение проводилось с перебором случайных инициализаций; решение с нулевой ошибкой найдено с первой попытки (seed = 0). Итоговые параметры: W = [[0.398, −0.770], [1.106, 1.290]], b = [0.347, 0.292], MAE = 0.000. Каждый нейрон задаёт свою разделяющую прямую; их пересечение делит плоскость на четыре области, соответствующие классам.

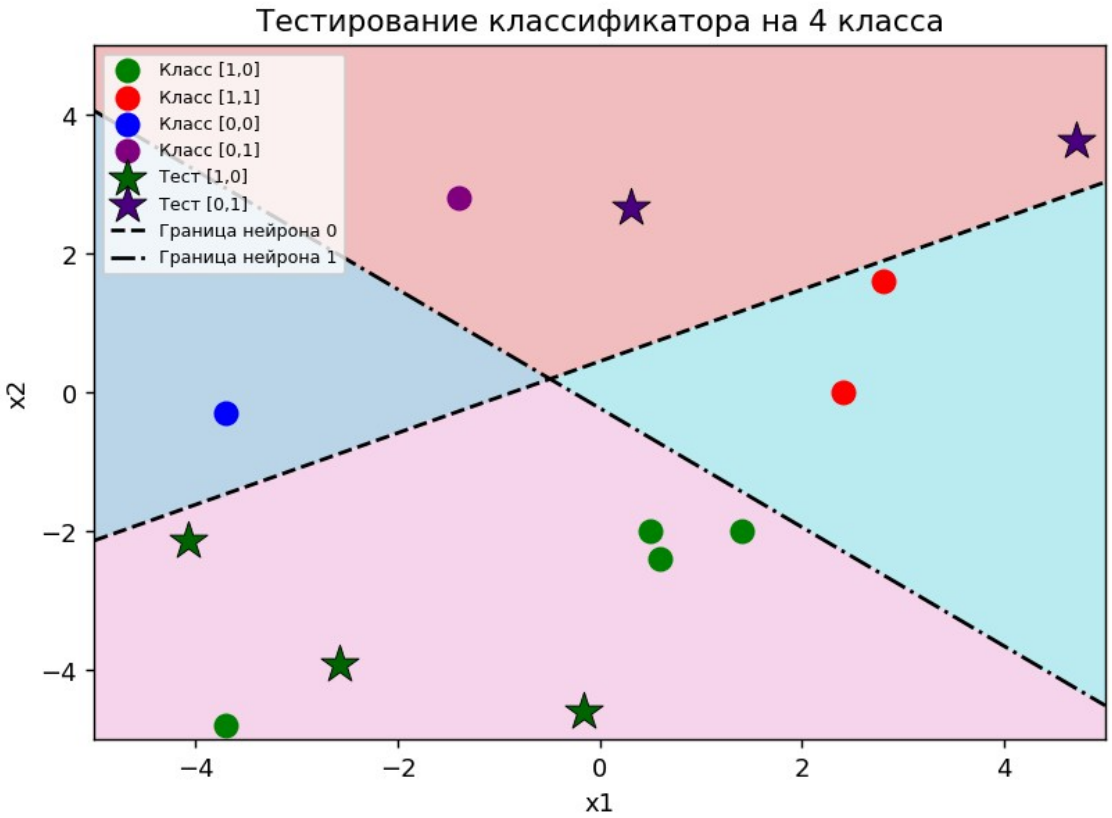


Рис. 6 — Тестирование классификатора на 4 класса (5 контрольных точек)

Код программы

```
import numpy as np
import matplotlib.pyplot as plt

# Вспомогательные функции

def hardlim(n):
    return (n >= 0).astype(float)

def predict(X, W, b):
    return hardlim(W @ X + b.reshape(-1, 1))

def mae(x):
    return np.mean(np.abs(x))

def rosenblatt_train(X, y, W, b, epochs=50, lr=0.5):
    n_samples = X.shape[1]
    for ep in range(epochs):
        for j in range(n_samples):
            xi = X[:, j : j + 1]
            yi = y[:, j : j + 1]
            y_pred = hardlim(W @ xi + b.reshape(-1, 1))
            error = yi - y_pred
            if mae(error) != 0:
                W = W + lr * (error @ xi.T)
                b = b + lr * error.flatten()
        if mae(y - predict(X, W, b)) == 0:
            break
    return W, b

X_train = np.array([[0.7, -2.7, 3.2, -4.6, 1.4, 1.4],
                    [-4.5, -1.5, -4.9, -3.4, 2.3, -0.5]])
y_train = np.array([[0.0, 1.0, 0.0, 1.0, 1.0, 0.0]])

print("X_train:", X_train)
print("y_train:", y_train)

np.random.seed(12)
W1 = np.random.uniform(-1, 1, (1, 2))
b1 = np.random.uniform(-1, 1, 1)

print("W начальные:", W1)
print("b начальное:", b1)

lr = 0.5
n_samples = X_train.shape[1]

for ep in range(1, 3):
    for j in range(n_samples):
```

```
        xi = X_train[:, j : j + 1]
        yi = y_train[:, j : j + 1]
        y_pred = hardlim(W1 @ xi + b1.reshape(-1, 1))
        error = yi - y_pred
        if mae(error) != 0:
            W1 = W1 + lr * (error @ xi.T)
            b1 = b1 + lr * error.flatten()
    y_out = predict(X_train, W1, b1)
    print(f"Эпоха {ep}: W={W1}, b={b1}, MAE={mae(y_train - y_out):.4f}")

x1_range = np.linspace(-5, 5, 200)
x2_line = -(W1[0, 0] * x1_range + b1[0]) / W1[0, 1]

xx1, xx2 = np.meshgrid(np.linspace(-5, 5, 200), np.linspace(-5, 5, 200))
grid_points = np.vstack([xx1.ravel(), xx2.ravel()])
grid_classes = predict(grid_points, W1, b1).reshape(xx1.shape)

fig, ax = plt.subplots()
ax.pcolormesh(xx1, xx2, grid_classes, alpha=0.3, cmap="coolwarm")
colors = {0.0: "blue", 1.0: "red"}
labels = {0.0: "Класс 0", 1.0: "Класс 1"}
plotted = set()
for i in range(X_train.shape[1]):
    cls = y_train[0, i]
    label = labels[cls] if cls not in plotted else None
    ax.scatter(X_train[0, i], X_train[1, i], color=colors[cls],
label=label, s=80, zorder=3)
    plotted.add(cls)
ax.plot(x1_range, x2_line, "k-", label="Разделяющая прямая")
ax.set_xlim(-5, 5)
ax.set_ylim(-5, 5)
ax.set_xlabel("x1")
ax.set_ylabel("x2")
ax.set_title("После ручного обучения (2 эпохи)")
ax.legend()
plt.tight_layout()
plt.show()

np.random.seed(15)
W1 = np.random.uniform(-1, 1, (1, 2))
b1 = np.random.uniform(-1, 1, 1)

W1, b1 = rosenblatt_train(X_train, y_train, W1, b1, epochs=50, lr=0.5)

y_out = predict(X_train, W1, b1)
print("W финальные:", W1)
print("b финальное:", b1)
print(f"MAE на обучающей выборке: {mae(y_train - y_out):.4f}")

np.random.seed(33)
X_test = np.random.uniform(-5, 5, (2, 3))
```

```
y_test = predict(X_test, W1, b1)

for i in range(3):
    print(f"Точка {i+1}: x1={X_test[0,i]:.3f}, x2={X_test[1,i]:.3f} ->
класс {int(y_test[0,i])}")

xx1, xx2 = np.meshgrid(np.linspace(-5, 5, 200), np.linspace(-5, 5, 200))
grid_points = np.vstack([xx1.ravel(), xx2.ravel()])
grid_classes = predict(grid_points, W1, b1).reshape(xx1.shape)

x1_range = np.linspace(-5, 5, 200)
x2_line = -(W1[0, 0] * x1_range + b1[0]) / W1[0, 1]

fig, ax = plt.subplots()
ax.pcolormesh(xx1, xx2, grid_classes, alpha=0.3, cmap="coolwarm")

colors_train = {0.0: "blue", 1.0: "red"}
labels_train = {0.0: "Класс 0 (обучение)", 1.0: "Класс 1 (обучение)"}
plotted = set()
for i in range(X_train.shape[1]):
    cls = y_train[0, i]
    label = labels_train[cls] if cls not in plotted else None
    ax.scatter(X_train[0, i], X_train[1, i], color=colors_train[cls],
label=label, s=80, zorder=3)
    plotted.add(cls)

colors_test = {0.0: "cyan", 1.0: "orange"}
labels_test = {0.0: "Тест, класс 0", 1.0: "Тест, класс 1"}
plotted_test = set()
for i in range(X_test.shape[1]):
    cls = y_test[0, i]
    label = labels_test[cls] if cls not in plotted_test else None
    ax.scatter(X_test[0, i], X_test[1, i], color=colors_test[cls],
marker="*",
                s=200, zorder=4, edgecolors="black", linewidths=0.5,
label=label)
    plotted_test.add(cls)

ax.plot(x1_range, x2_line, "k-", label="Разделяющая прямая")
ax.set_xlim(-5, 5)
ax.set_ylim(-5, 5)
ax.set_xlabel("x1")
ax.set_ylabel("x2")
ax.set_title("Тестирование бинарного классификатора")
ax.legend()
plt.tight_layout()
plt.show()

# Добавляем точку (-3.0, 1.0) класса 0 в область класса 1 — нарушаем
линейную разделимость
```

```
X_mod = np.hstack([X_train, np.array([[-3.0], [1.0]])])
y_mod = np.hstack([y_train, np.array([[0.0]])])

print("X_mod:", X_mod)
print("y_mod:", y_mod)

np.random.seed(20)
W_mod = np.random.uniform(-1, 1, (1, 2))
b_mod = np.random.uniform(-1, 1, 1)

W_mod, b_mod = rosenblatt_train(X_mod, y_mod, W_mod, b_mod, epochs=50,
lr=0.5)

y_out_mod = predict(X_mod, W_mod, b_mod)
print("W финальные:", W_mod)
print("b финальное:", b_mod)
print(f"MAE на модифицированной выборке: {mae(y_mod - y_out_mod):.4f}")

x1_range = np.linspace(-5, 5, 200)
x2_line_mod = -(W_mod[0, 0] * x1_range + b_mod[0]) / W_mod[0, 1]

xx1, xx2 = np.meshgrid(np.linspace(-5, 5, 200), np.linspace(-5, 5, 200))
grid_points = np.vstack([xx1.ravel(), xx2.ravel()])
grid_classes_mod = predict(grid_points, W_mod, b_mod).reshape(xx1.shape)

fig, ax = plt.subplots()
ax.pcolormesh(xx1, xx2, grid_classes_mod, alpha=0.3, cmap="coolwarm")

colors_cls = {0.0: "blue", 1.0: "red"}
labels_cls = {0.0: "Класс 0", 1.0: "Класс 1"}
plotted = set()
for i in range(X_mod.shape[1]):
    cls = y_mod[0, i]
    label = labels_cls[cls] if cls not in plotted else None
    ax.scatter(X_mod[0, i], X_mod[1, i], color=colors_cls[cls],
label=label, s=80, zorder=3)
    plotted.add(cls)

ax.plot(x1_range, x2_line_mod, "k-", label="Разделяющая прямая")
ax.set_xlim(-5, 5)
ax.set_ylim(-5, 5)
ax.set_xlabel("x1")
ax.set_ylabel("x2")
ax.set_title("Линейно неразделимый случай (MAE > 0)")
ax.legend()
plt.tight_layout()
plt.show()

X_train2 = np.array([[0.6, 2.4, 1.4, -3.7, -1.4, 2.8, -3.7, 0.5],
                     [-2.4, 0, -2, -0.3, 2.8, 1.6, -4.8, -2]])
```

```
y_train2 = np.array([[1.0, 1.0, 1.0, 0.0, 0.0, 1.0, 1.0, 1.0],
                     [0.0, 1.0, 0.0, 0.0, 1.0, 1.0, 0.0, 0.0]])

print("X_train2:", X_train2)
print("y_train2:", y_train2)

print("\n=== Структура сети ===")
print(f"Количество входов: 2")
print(f"Количество нейронов: 2")
print(f"Функция активации: hardlim")
print(f"Смещение (bias): да")

best_mae = 1.0
best_W, best_b = None, None

max_attempts = 100
for attempt in range(max_attempts):
    np.random.seed(attempt)
    W2 = np.random.uniform(-1, 1, (2, 2))
    b2 = np.random.uniform(-1, 1, 2)
    W2, b2 = rosenblatt_train(X_train2, y_train2, W2, b2, epochs=50,
lr=0.5)
    y_out2 = predict(X_train2, W2, b2)
    current_mae = mae(y_train2 - y_out2)
    if current_mae < best_mae:
        best_mae = current_mae
        best_W, best_b = W2.copy(), b2.copy()
    if current_mae == 0:
        print(f"Сошлось за попытку {attempt + 1} (seed={attempt})")
        break

W2, b2, final_mae = best_W, best_b, best_mae

if final_mae != 0:
    print(f"Не удалось достичь MAE=0 за {max_attempts} попыток.")

print("W2 финальные:", W2)
print("b2 финальные:", b2)
print(f"MAE на обучающей выборке: {final_mae:.4f}")

np.random.seed(55)
X_test2 = np.random.uniform(-5, 5, (2, 5))
y_test2 = predict(X_test2, W2, b2)

for i in range(5):
    print(f"Точка {i+1}: x1={X_test2[0,i]:.3f}, x2={X_test2[1,i]:.3f} ->
метка [{int(y_test2[0,i])}, {int(y_test2[1,i])}]")

xx1, xx2 = np.meshgrid(np.linspace(-5, 5, 200), np.linspace(-5, 5, 200))
grid_points = np.vstack([xx1.ravel(), xx2.ravel()])
```

```
grid_out = predict(grid_points, W2, b2)
grid_label = (grid_out[0] * 2 + grid_out[1]).reshape(xx1.shape)

fig, ax = plt.subplots()
ax.pcolormesh(xx1, xx2, grid_label, alpha=0.3, cmap="tab10", vmin=0,
vmax=3)

class_colors = {(1, 1): "red", (0, 0): "blue", (1, 0): "green", (0, 1):
"purple"}
class_labels = {(1, 1): "Класс [1,1]", (0, 0): "Класс [0,0]",
                (1, 0): "Класс [1,0]", (0, 1): "Класс [0,1]"}
plotted_train = set()
for i in range(X_train2.shape[1]):
    cls = (int(y_train2[0, i]), int(y_train2[1, i]))
    label = class_labels[cls] if cls not in plotted_train else None
    ax.scatter(X_train2[0, i], X_train2[1, i], color=class_colors[cls],
s=80, zorder=3, label=label)
    plotted_train.add(cls)

test_colors = {(1, 1): "darkred", (0, 0): "darkblue", (1, 0): "darkgreen",
(0, 1): "indigo"}
test_labels = {(1, 1): "Тест [1,1]", (0, 0): "Тест [0,0]",
               (1, 0): "Тест [1,0]", (0, 1): "Тест [0,1]"}
plotted_test = set()
for i in range(X_test2.shape[1]):
    cls = (int(y_test2[0, i]), int(y_test2[1, i]))
    label = test_labels[cls] if cls not in plotted_test else None
    ax.scatter(X_test2[0, i], X_test2[1, i], color=test_colors[cls],
marker="*",
               s=250, zorder=4, edgecolors="black", linewidths=0.5,
label=label)
    plotted_test.add(cls)

x1_range = np.linspace(-5, 5, 200)
for i, (ls, lbl) in enumerate([("k--", "Граница нейрона 0"), ("k-.",
"Граница нейрона 1")]):
    if abs(W2[i, 1]) > 1e-9:
        x2_line = -(W2[i, 0] * x1_range + b2[i]) / W2[i, 1]
        ax.plot(x1_range, x2_line, ls, label=lbl)

ax.set_xlim(-5, 5)
ax.set_ylim(-5, 5)
ax.set_xlabel("x1")
ax.set_ylabel("x2")
ax.set_title("Тестирование классификатора на 4 класса")
ax.legend(loc="upper left", fontsize=7)
plt.tight_layout()
plt.show()
```

Вывод

В ходе работы исследованы основные свойства персептрона Розенблатта — простейшей однослойной нейронной сети с пороговой функцией активации hardlim — и его применение к задаче классификации точек на плоскости. Рассмотрены линейно разделимая и линейно неразделимая постановки, а также обобщение модели на несколько классов.

На первом этапе однонейронный персептрон обучен разделять два линейно разделимых класса. При ручном обучении нулевая ошибка (MAE = 0) достигнута уже за две эпохи, при автоматическом — за несколько эпох при другой случайной инициализации; в обоих случаях построенная дискриминантная прямая корректно классифицирует контрольные точки. Это практически подтверждает работоспособность правила Розенблатта и теорему о сходимости персептрона для линейно разделимых данных.

На втором этапе в выборку добавлена точка, нарушающая линейную разделимость классов. В этом случае правило Розенблатта не сходится: веса циклически меняются и не стабилизируются, а итоговая ошибка остаётся ненулевой (MAE = 0.4286, неверно классифицируются три точки из семи). Это наглядно иллюстрирует фундаментальное ограничение однослойного персептрона — он способен решать только линейно разделимые задачи и не может построить нелинейную разделяющую границу.

На третьем этапе персептрон из двух нейронов обучен классифицировать точки на четыре класса с помощью двухбитного кодирования выхода; достигнута нулевая ошибка, а две разделяющие прямые корректно делят плоскость на четыре области, соответствующие классам. Тем самым показано, что увеличение числа нейронов в слое расширяет множество решаемых задач в пределах линейной разделимости. Все этапы работы выполнены, цель достигнута.